Architectural Evaluation, Open-Source Repository Mapping, and Multi-Agent Implementation Handover for an AI-Driven Audio Production Workstation

Executive Architectural Assessment: SoundCraft as the Core Foundation

Establishing an end-to-end, AI-augmented audio production and reverse-engineering workstation requires unifying low-latency digital signal processing (DSP), symbolic music representation, deep learning audio analysis, and an intuitive user interface where preliminary workspace specifications leave implementation details open[1]. Evaluating ArtCraft's **SoundCraft** (storytold/soundcraft, distributed via getartcraft.com/apps/soundcraft) confirms that the repository provides an ideal foundational base for both the end-user graphical interface and the underlying session execution engine[2]. Rebuilt from scratch in pure Rust (Edition 2024, requiring Rust 1.90) as a clean-room reimplementation of Avid Pro Tools, SoundCraft reached version 0.7.0 in October 2026 and has rapidly accumulated over 3,200 GitHub stars[2].
Unlike legacy open-source digital audio workstations (DAWs) such as Ardour or Audacity—which rely on monolithic C++ codebases, copyleft GPL licensing, and tightly coupled graphical event loops[6]—SoundCraft is architected as a modular Cargo workspace with strict dependency layering and permissive dual MIT OR Apache-2.0 licensing[3]. Crucially for a machine-learning-driven workflow, SoundCraft was designed from inception to be "agent-ready"[8]. Every editing, mixing, routing, and playback action is exposed simultaneously to human operators via the graphical interface and to autonomous models via a headless command-line interface (soundcraft-cli), a bidirectional JSON control channel, and a built-in Model Context Protocol (MCP) server (soundcraft-automation)[3].

RUST CARGO WORKSPACE | MIT / Apache-2.0

SoundCraft Combines a Strict Six-Layer Rust Architecture with Native **MCP Agent Control** at 40% Pro Tools Parity

Release Status	DAW Parity	GUI Performance	Binary Footprint	GitHub Adoption
v0.7.0	~40%	120 FPS	10.9 MB	3.2k ★
Pre-alpha (Oct 2026)	Avid workflow match	at 70+ audio tracks	Win x64 installer .msi	36 watchers
Clean-room Pro Tools	Core Edit & Mix live	egui desktop & WebAudio	Zero bloat / lean Rust	storytold/soundcraft

Cargo Workspace: Strict 6-Layer Modular Stack

Click or hover any crate layer to inspect responsibilities, safety guarantees, and data flows.

Safe Rust · MCP / AI Agent · Isolated FFI (Unsafe)

APPS | soundcraft (Desktop DX12/Metal) | soundcraft-cli | soundcraft-web (WASM)

L6 soundcraft-ui-egui — 120 FPS GUI

Movable panels: Edit/Mix windows, Tracks & Groups, Waveforms, Memory Locations.

L5 soundcraft-automation — AI Agent Endpoint — MCP Server

Model Context Protocol server + control client. Errors return structured values, never crash.

L4 soundcraft-engine & soundcraft-playback — CoreAudio · WASAPI · WebAudio

Session editing commands, non-destructive undo, hardware device abstraction, offline bounce.

L3 soundcraft-mix — Parallel DAG Mixer

Stereo to 9.1.6 / Ambisonics, divergence, height panner, live speaker meters, dither & norm.

L1-2 soundcraft-{time, audio-io, midi, video, dsp} — Pure DSP & Codecs

Timebases/tempo map, peak file overview, SMF parser, built-in synths, FFT & LUFS loudness.

ISOLATED FFI — Unsafe C-ABI

External Host Crates

clap-host	Native
vst3-host	Active Bugs
au-host	Issue #640

Defensive Architecture: Created off audio-thread to guard realtime engine from external crashes.

L5 · Native MCP Server & Control Client — Safe Rust (No Panic on Bad Agent Call)

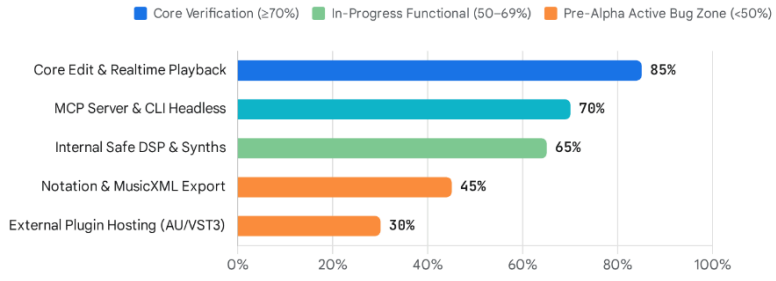
Exposes Model Context Protocol (MCP) tool calls and JSON-RPC control channels. Enables AI assistants and CLI scripts to query sessions, insert tracks, automate faders, and trigger stem bounces without GUI overhead.

Path: `crates/soundcraft-automation`

Flow: `Upstream: Dispatches to L4 Engine · Downstream: Feeds state diffs to L6 UI`

Subsystem Production Readiness & Known Pre-Alpha Defect Density

Evaluation basis: v0.7.0 Releases & GitHub Issue Log



Verified Functional: WebAudio 48kHz stereo demo, 120 FPS timeline renderer, parallel offline mix bounce, MCP agent JSON tool calls.

Active Defect Focus: VST3 automation scaling normalized to 0-1 (#599/#600), AU plugin crash on load (#640), Suno remix credit timeouts (#654).

Architectural breakdown of storytold/soundcraft as of version 0.7.0 (October 2026). Strict layering ensures zero dependencies on the user interface below Layer 6, enabling headless CLI and Model Context Protocol (MCP) execution alongside 120 FPS desktop and WebAssembly rendering.

Data sources: GitHub (storytold/soundcraft), ArtCraft Tutorial, OpenSrcs

Architectural Strengths of the SoundCraft Workspace

The primary engineering advantage of adopting storytold/soundcraft lies in its strict separation of concerns across internal workspace crates[3]. Nothing below the Layer 6 user interface crate (soundcraft-ui-egui) has any dependency on egui or windowing primitives, meaning the user interface could be swapped or augmented without altering a single line of audio or session logic[3]. Consequently, the entire audio engine compiles cleanly into a standalone desktop binary (apps/soundcraft), a headless CLI daemon (apps/soundcraft-cli) for cloud or local agent pipelines, or a WebAssembly bundle (apps/soundcraft-web) that runs directly in a browser tab at 48 kHz stereo via WebAudio[2].

Furthermore, SoundCraft enforces rigorous systems-engineering invariants documented in its repository guidelines (AGENTS.md and CLAUDE.md, derived from storytold/craftrules)[11]. First, the codebase guarantees zero-panic execution on malformed input[3]. All numerical parameters derived from external files, CLI arguments, or MCP agent calls are treated as potentially hostile[12]. Errors are propagated strictly as typed values, ensuring that a corrupted media file or an out-of-bounds language model tool call produces a structured diagnostic message rather than crashing the workstation[3]. Second, real-time audio thread isolation is strictly preserved: the audio callback thread never allocates heap memory, blocks on mutexes, or performs disk or console I/O[3]. Even diagnostic logging (apps/soundcraft/src/logging.rs) is lock-free, capturing the first log record without waiting or allocating, incrementing an atomic counter for subsequent events, and delegating file flushing—capped at 16 MiB with automatic rotation across soundcraft.1.log and soundcraft.2.log—to the UI thread[3]. Similarly, third-party CLAP, VST3, and Audio Unit plugin instances are instantiated off the audio thread and isolated inside dedicated unsafe crates (soundcraft-clap-host, soundcraft-vst3-host, soundcraft-au-host) behind the safe soundcraft-dsp Plugin trait[3]. Finally, interactive playback and offline stem or mix bounces share the exact same parallel mixing engine (soundcraft-mix), supporting ten inserts and ten sends per track, VCA masters, routing folders, automatic plugin delay compensation, surround panning up to 9.1.6 and Ambisonics, and integrated loudness metering in LUFS and True Peak[3].

Pre-Alpha Maturity Caveats and Operational Realities

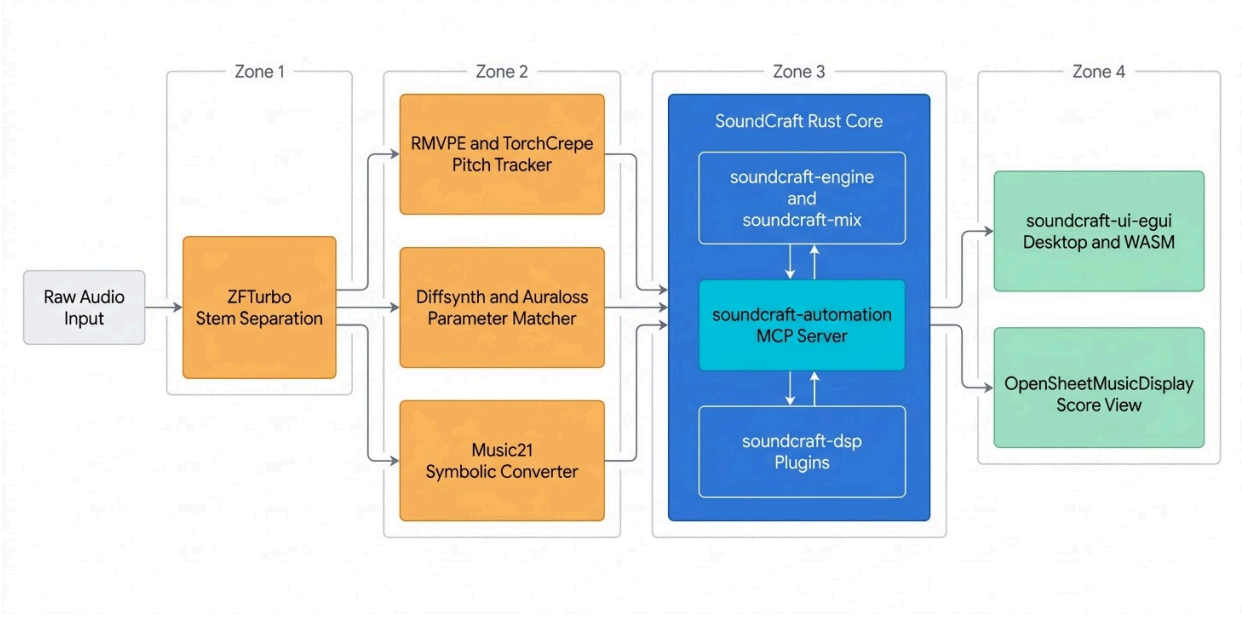
While SoundCraft's development velocity is extraordinary—driven by heavy utilization of AI coding agents alongside community contributors, a methodology that has sparked intense debate across audio engineering forums such as KVR Audio and Linux Audio communities[7]—system architects must account for its **pre-alpha** maturity[2]. The ArtCraft development team estimates that SoundCraft is currently approximately 40% of the way toward full functional parity with Avid Pro Tools, with granular feature tracking maintained across ROADMAP.md, docs/target-app-parity.md, docs/gaps.md, and docs/parity-checklist.md[3].

Independent verification of builds v0.6.1 and v0.7.0 confirms that while the Edit window, timeline transport, built-in synthesized demo playback (Midnight Groove), and WebAssembly

audio device handoff work reliably out of the box[2], several peripheral subsystems exhibit active pre-alpha defects reflected in the repository's issue tracker[19]. Specifically, while built-in DSP plugins (7-band and 1-band EQ, compressor/limiter, gate, de-esser, maximizer, channel strip, reverbs, delay, chorus, phaser, saturator, pitch/time shift, dither, synth, drum synth, and sampler) operate natively and deterministically inside soundcraft-dsp[2], external VST3 and Audio Unit hosts have open bug reports regarding VST3 parameter ranges displaying strictly as 0–1 (#599), broken VST3 parameter automation (#600), UI redraw artifacts (#598), and macOS Audio Unit Cocoa view initialization issues (#640, actively targeted in PR #687)[19]. In the import/export subsystem, bounce operations support WAV, AIFF, and FLAC at 16/24/32-bit float, but native MP3 bounce export is not yet implemented (#648)[3]. Users have also reported occasional export stalls at 95% (#624, #665) and video track import issues (#658), alongside active pull requests fixing AAC/MP4 edit-list encoder delay padding (#713) and AC-3/E-AC-3 format reporting (#714)[19]. Recording live MIDI directly from external hardware controllers and exchanging studio sessions via AAF/OMF containers also remain unimplemented[7]. On Windows desktops, the renderer initializes DirectX 12 using the built-in FXC shader compiler by default so dxcompiler.dll is unnecessary, requiring environment overrides ($env:WGPU_DX12_COMPILER = "dxc") only when troubleshooting specific GPU driver configurations[3].

Open-Source Repository Mapping by Functional Subsystem

Because SoundCraft is a pure Rust DAW and session host rather than a deep learning training framework, specialized audio intelligence tasks—such as polyphonic source separation, neural pitch tracking, differentiable synthesizer sound matching (parameter estimation), and web-native sheet music engraving—must be covered by integrating domain-specific open-source GitHub repositories[3]. Rather than bloating SoundCraft's real-time Rust binary with heavy C++ or Python machine learning runtimes, these repositories should be orchestrated as modular sidecar services or frontend components that read and write session artifacts via SoundCraft's CLI and MCP automation layer[3]. This architectural pattern is already validated in the SoundCraft ecosystem, exemplified by community PR #676 adding an Edit > VO Cut Pipeline for Whisper + LLM voiceover cleanup[20].



Audio Source Separation and Stem Deconstruction

When an end user imports a mixed stereo recording to reverse-engineer or remix, the audio must first be decomposed into isolated constituent stems—vocals, bass, drums, synthesizers, and sound effects—before pitch detection or synthesizer sound matching can succeed. The primary repository to cover this subsystem is ZFTurbo/Music-Source-Separation-Training, which provides a comprehensive PyTorch training and inference framework for state-of-the-art music demixing models[21]. Notably, this framework includes the Bandit Plus architecture, which performs reliably at separating complex sound effects alongside musical stems, and ships with extensive pre-trained model weights (docs/pretrained_models.md) as well as reference GUI integration code (docs/gui.md)[26]. For workflows requiring custom model experimentation on specialized multi-channel or mono acoustic datasets, asteroid-team/asteroid (cataloged in Yuan-ManX/audio-development-tools) serves as a secondary PyTorch source separation toolkit[27].

Polyphonic Pitch Estimation and Audio-to-MIDI Transcription

Once stems are isolated, converting melodic and harmonic audio into editable MIDI clips and score notation requires high-precision fundamental frequency (F_0) estimation. While SoundCraft includes a basic built-in Audio-to-MIDI pitch detection routine within its MIDI workflow[4], polyphonic bleed or expressive vocal and lead lines benefit substantially from dedicated deep neural pitch trackers. For vocal and polyphonic lead lines, Dream-High/RMVPE (*RMVPE: A Robust Model for Vocal Pitch Estimation in Polyphonic Music*) is the premier open-source choice under an Apache-2.0 license[24]. RMVPE extracts vocal pitches directly from polyphonic mixtures with superior accuracy and noise robustness across all signal-to-noise ratio (SNR) levels compared to earlier trackers[28]. Its evaluation pipeline (evaluate.py) segments audio into 128-step slices and converts predicted pitch probability distributions into frequency

contours using either local average cents (to_local_average_cents) or Viterbi decoding (to_viterbi_cents)[29]. For monophonic instrumental stems and direct Differentiable Digital Signal Processing (ctralie/ddsp) resynthesis, maxrmorrison/torchcrepe (alongside gudgud96/torchcrepeV2) provides a battle-tested PyTorch implementation of the CREPE pitch tracker with over 20 million downloads[30].

Synthesizer Sound Matching, Parameter Estimation, and Generative Conditioning

A core capability of intelligent audio production is **synthesizer sound matching**—also referred to in literature as automatic synthesizer programming (nASP) or synthesizer parameter estimation—which approximates an arbitrary target sound by automatically inferring the exact oscillator, filter, and envelope parameters of a synthesizer[22]. For subtractive and additive-subtractive synthesis, hyakuchiki/Diffsynth and SSSSM-DDSP (*Semi-Supervised Synthesizer Sound Matching with Differentiable DSP* by Naotake Masuda and Daisuke Saito) provide a PyTorch differentiable synthesizer inspired by Image-Line Harmor and Native Instruments Razor[22]. Crucially, Diffsynth overcomes the out-of-domain generalization barrier—matching real-world target sounds where ground-truth synthesis parameters are unavailable—via a three-stage training curriculum[22]. First, an estimator neural network takes frame-by-frame Mel-spectrograms and pre-trains on synthetic in-domain data using parameter loss (P-loss) for 50 epochs[22]. Second, over the next 150 epochs, a spectral loss computed through the differentiable synthesizer is linearly introduced to replace the parameter loss[22]. Finally, the network is fine-tuned using spectral loss on real-world out-of-domain audio, with sigmoid nonlinearity normalizing all predicted synthesis parameters to $(0, 1)$ [22]. Where target synthesizers are non-differentiable, the authors' companion Quality-Diversity (QD) evolutionary algorithm searches the parameter space to yield diverse, high-performing patch matches, while ensemble programmer architectures (evaluating structural templates across UGen combinations via MFCC distance) further improve out-of-domain match accuracy[34].
For Frequency Modulation (FM) synthesis and complex commercial presets, gwendal-lv/preset-gen-vae (developed by Gwendal Le Vaillant) and Sound2Synth represent the state of the art in Yamaha DX7 FM synthesizer sound matching and continuous latent sound morphing[27]. Furthermore, to bridge non-differentiable third-party synthesizers without rewriting their DSP in PyTorch, neural proxy preset encoders such as HN-PTGRU tokenize numerical parameters ($x_i \in [0, 1]$), binary switches ($x_i \in \{0, 1\}$), and categorical routing integers ($x_k \in \{1, \dots, K\}$) via a learnable Preset Tokenizer ($d_{\text{in}} = N_s \cdot d_{\text{token}}$), aligning preset embeddings $f_\theta(\bar{x})$ directly with pretrained audio model embeddings $g_\phi(s(\bar{x}, \bar{\omega}))$ [36].
Training and optimizing these sound-matching networks relies on csteinmetz1/auraloss (by Christian J. Steinmetz and Joshua D. Reiss), which provides a unified PyTorch library of time-domain, frequency-domain, and perceptually weighted audio loss functions[39]. Its

MultiResolutionSTFTLoss combines Frobenius-norm spectral convergence

$$\mathrm{SC}(y, \hat{y}) = \frac{\| \, |\mathrm{STFT}(y)| - |\mathrm{STFT}(\hat{y})| \, \|_F}{\| \, |\mathrm{STFT}(y)| \, \|_F}$$

and L_1 log-magnitude distance across multiple window resolutions, alongside MelSTFTLoss, RandomResolutionSTFTLoss (which randomizes STFT window and hop sizes to prevent resolution bias), SumAndDifferenceSTFTLoss ($y_{\text{sum}} = y_{\text{left}} + y_{\text{right}}$, $y_{\text{diff}} = y_{\text{left}} - y_{\text{right}}$ for stereo mid/side invariance), and time-domain ESRLoss and LogCoshLoss with FIR perceptual pre-emphasis filters[39]. When a target sound or musical passage requires generative diffusion rather than parametric synthesis, modelscope/DiffSynth-Studio supplies full training, LoRA adaptation, and low-VRAM inference pipelines—incorporating modelscope/entropack arbitrary-bitrate weight quantization and layer-level disk offload—for DiffSynth-Music (which injects composable audio conditioning via layer-wise KV-cache adapters), ACE-Step (ACE-Step/acestep-v15-xl-sft), MiniMax-Music3, and YuE2[41].

Symbolic Musicology, MusicXML Conversion, and Sheet Music Engraving

While SoundCraft includes an internal Score Editor for MIDI tracks and native MusicXML export[3], rich web-based score interactivity and symbolic musicological conversion are best served by pairing opensheetmusicdisplay/opensheetmusicdisplay (OSMD) and music21[23]. Maintained in TypeScript atop VexFlow under a permissive BSD-3-Clause license, OSMD (reaching v2.2.1 on October 12, 2026) renders MusicXML in browsers and headlessly in Node.js[23]. Recent releases between v2.0.0 and v2.2.1 replaced canvas pixel reads (getImageData) with cached geometric skyline outline calculations—achieving a 1.8x to 4x rendering speedup—and added incremental system-by-system lazy rendering (renderNext()), cross-staff slurs, accurate dynamics and wedge alignment, Jianpu numbered notation (v2.2.1), and a dedicated React wrapper (react-opensheetmusicdisplay)[23]. On the backend, ianberman/MusicXML-to-MIDI-Converter (built on MIT's music21 in Python 3.10+) handles programmatic MusicXML-to-MIDI conversion and staff-level object dictionary manipulation[44].

Declarative Web DSP and Algorithmic Pattern Live Coding

For extensions requiring custom programmable DSP plugins or live-coded pattern generation alongside SoundCraft, elemaudio/elementary (elemaudio/offline-examples, elemaudio/srvb, and cristianvogel/srvb-plugin-svelte) offers an MIT-licensed declarative JavaScript/C++ audio framework[27]. Elementary decouples functional JavaScript state descriptions from its underlying C++/WASM runtime, enabling identical DSP graphs to run in Node.js CLI scripts, browser tabs, or native WebView audio plugins built with SvelteKit or React[48]. For algorithmic pattern composition, tidalcycles/strudel (archived on GitHub in June 2025 and actively

developed at codeberg.org/uzu/strudel) ports Haskell's TidalCycles pattern algebra to JavaScript and WebAudio, complete with wavetable, supersaw, and ladder-filter synthesis[50].

Functional Subsystem	Repository Identifier	Primary Language & Stack	License	Integration Role with SoundCraft
Core DAW Engine, UI & MCP Host	storytold/soundcraft [cite: 3]	Rust (egui, wgpu, WASM)[3]	MIT OR Apache-2.0 (Marks restricted)[3]	Primary workstation host, real-time mixer, Edit/Mix UI, CLI & MCP server[3].
Stem & SFX Source Separation	ZFTurbo/Music-Source-Separation-Training [cite: 21]	Python (PyTorch)[21]	MIT / Open Source[21]	Pre-processing microservice splitting mixed audio into stems before track import[20].
Polyphonic Vocal Pitch (F_0)	Dream-High/RMVPE [cite: 24]	Python (PyTorch)[24]	Apache-2.0 [cite: 24]	Extracts noise-robust F_0 contours from polyphonic audio for MIDI conversion[28].
Monophonic Pitch & Periodicity	maxrmorrison/torchcrepe [cite: 30]	Python (PyTorch)[30]	MIT	Frame-level pitch and confidence extraction for DDSP and synth matching[31].
Differentiable Synth Matching	hyakuchiki/Diffsynth & SSSSM-DDSP [cite: 27, 35]	Python (PyTorch)[22]	Open Academic	Estimates synthesizer parameters from

				real-world target audio via spectral loss[22].
FM Synth Matching & Morphing	gwendal-lv/preset-gen-vae [cite: 38]	Python (PyTorch)[38]	Open Source	Infers multi-operator FM synth patches and interpolates latent presets[38].
Perceptual Audio Losses	csteinmetz1/auraloss [cite: 39]	Python (PyTorch)[39]	Apache-2.0	Multi-resolution STFT, Mel-STFT, and stereo sum/difference loss computation[39].
Generative Music Conditioning	modelscope/DiffSynth-Studio [cite: 41]	Python (PyTorch, EntroPack)[41]	Apache-2.0	Hosts DiffSynth-Music KV-cache audio conditioning and ACE-Step generation[41].
Sheet Music Engraving (Web)	opensheetmusicdisplay/opensheetmusicdisplay [cite: 23]	TypeScript (VexFlow, React)[23]	BSD-3-Clause [cite: 23]	Renders MusicXML exported by SoundCraft into interactive browser scores[3].
Symbolic Musicology & MIDI	cuthbertLab/music21 / ianberman/MusicXML-to-MIDI-Converter	Python 3.10+[44]	BSD / Open Source	Converts and validates MusicXML and Standard MIDI Files across

	[cite: 44]			pipelines[44].
Declarative DSP Graph	elemaudio/elementary [cite: 27, 48]	JavaScript / C++ / WASM[48]	MIT [cite: 49]	Declarative DSP rendering for custom web plugins or offline Node.js processing[48].
Algorithmic Pattern Engine	codeberg.org/uzu/strudel (tidalcycles/strudel)[50]	JavaScript (WebAudio)[51]	AGPL-3.0 [cite: 50]	Optional pattern live-coding generator (must remain isolated due to AGPL-3.0)[50].

End-User UI Strategy: Native Egui vs. Hybrid WebAssembly Frontends

Delivering a responsive, studio-grade user interface on top of SoundCraft can follow either a pure native desktop deployment or a hybrid browser-hosted architecture, as soundcraft-ui-egui compiles cleanly to both targets from a single codebase[2].

Path A: Native Desktop Workstation (apps/soundcraft)

Compiling apps/soundcraft produces a compact native executable (~10.9 MB MSI installer on 64-bit Windows, alongside portable builds and native packages for macOS Universal, Linux AppImage/Flatpak/DEB/RPM on x86_64 and ARM64, and FreeBSD) with zero Electron or web-view memory overhead[2]. Between versions 0.6.1 and 0.7.0, the UI incorporated a shared docking manager (#167) where Tracks and Groups, Clips, MIDI Editor, Memory Locations, and Undo History operate as movable panels that users can drag to target edges, tab-group, or float within the main application window[3]. Dedicated Edit and Mix windows retain independent arrangements toggled via Cmd+=, while single-key Commands Focus shortcuts (F1–F4 for Shuffle, Slip, Spot, and Grid edit modes; F5–F10 for tools; Cmd+E to separate clips; Cmd+D to duplicate; Cmd+H to heal clips; Shift+Cmd+K to export clips) mirror professional studio ergonomics[3]. The immediate-mode egui renderer sustains 120 frames per second with over seventy tracks playing simultaneously, utilizing adaptive meter polling and repaint rates (#207), automatic idle sleep when occluded (#192), and smooth trackpad pinch-and-scroll zooming (#56)[3]. Custom AI functionality—such as stem decomposition or synthesizer patch matching—can be integrated directly into the native menu bar and dockable panel system in crates/ui-egui, following the exact blueprint of PR #676 (Edit > VO Cut Pipeline)[3].

Path B: Browser-Hosted Hybrid Studio (apps/soundcraft-web + TypeScript Host)

Because apps/soundcraft-web compiles the Rust engine and egui interface to WebAssembly via trunk serve --release[3], teams requiring zero-install browser delivery or rich web overlays can embed the SoundCraft WASM canvas inside a React or SvelteKit host application[23]. As of PR #158, the browser build supports opening, importing, saving, and exporting session and audio files directly in the browser tab while streaming 48 kHz stereo audio through WebAudio[2]. In this hybrid layout, the WASM viewport handles high-frame-rate timeline editing and mixing, while an adjacent DOM pane mounts opensheetmusicdisplay (OSMD v2.2.1) via react-opensheetmusicdisplay to provide responsive, screen-reader-accessible sheet music engraving synchronized with the DAW playhead[3].

Multi-Agent Implementation Handover Brief

The following technical specification serves as a direct, actionable engineering brief for autonomous coding models and implementation agents tasked with assembling the system.

System Invariants and Repository Governance Rules

All coding agents operating within the storytold/soundcraft workspace must read CLAUDE.md and AGENTS.md (which enforce storytold/craftrules) prior to generating code[3]. Agents must preserve the strict six-layer crate hierarchy: no crate below soundcraft-ui-egui may import UI dependencies, and every state-mutating feature must be registered as a deterministic command inside soundcraft-engine so that it is simultaneously reachable from the GUI menus, keyboard shortcuts, soundcraft-cli, the JSON control channel, and the MCP server[3].
Every command handler must treat input-derived numbers from files, CLI flags, or MCP JSON payloads as hostile, explicitly rejecting missing, non-numeric, NaN, or out-of-bounds values with typed Result::Err responses rather than panicking or silently clearing state (matching the defensive patterns established in PR #695, PR #703, and PR #705)[3]. Real-time audio threads inside soundcraft-mix and soundcraft-playback must never allocate memory, lock mutexes, or perform I/O; mix buffers must be reused across render blocks (PR #208), and all external ML inference or plugin instantiation must occur on background worker threads[3]. Finally, in strict compliance with docs/brand/LICENSE-brand.txt, agents preparing a derived build or fork must remove all ArtCraft and SoundCraft trademarks, logos, and icons from docs/brand/ and replace all product branding while retaining plain-text attribution in NOTICE and ATTRIBUTION.md[3].

Operational Task	Exact CLI / Agent Command	Execution Context & Governance Purpose
Run Native Desktop App	cargo run --release -p soundcraft -- --demo	Compiles and launches the native desktop DAW with

	[cite: 3]	the synthesized Midnight Groove demo session (libasound2-dev / alsa-lib-devel required on Linux)[2].
Serve WebAssembly Build	cd apps/soundcraft-web && trunk serve --release [cite: 3]	Compiles the WASM bundle and serves the browser studio at http://127.0.0.1:8080[3].
List All Engine Commands	soundcraft-cli commands [cite: 3]	Outputs the complete registry of dotted command IDs supported by soundcraft-engine and MCP[3].
Headless Scripted Execution	SOUNDCRAFT_NO_PREFS=1 soundcraft-cli run --demo --cmd 'session.audio_health={}' [cite: 3]	Executes deterministic engine commands headlessly without reading/writing user prefs or disk logs[3].
Bind MCP Server (Stdio)	claude mcp add soundcraft -- soundcraft-cli mcp --demo --automation-read-root ./in --automation-write-root ./out [cite: 3, 15]	Registers soundcraft-automation as an MCP server over stdio with strict directory sandboxing[15].
Bind MCP to Live GUI	soundcraft-cli mcp --connect PORT [cite: 12]	Bridges an external MCP agent directly to a running desktop UI instance via the JSON control channel[3].
Pre-Commit CI Verification	cargo xtask ci [cite: 3]	Runs formatting (rustfmt), clippy, unit tests, crates/automation/tests/agent_tasks.rs, layering checks, and WASM compilation[3].

Work Package A: Python ML Sidecar Service (Stem Separation, Pitch & Synth Matching)

Coding agents assigned to the machine learning backend must construct an isolated Python 3.10+ worker service that encapsulates ZFTurbo/Music-Source-Separation-Training, Dream-High/RMVPE, maxrmorrison/torchcrepe, csteinmetz1/auraloss, hyakuchiki/Diffsynth, and music21[21]. When invoked with an input audio file path and target workspace directory, the service first executes the Bandit Plus or HTDemucs separation pipeline from ZFTurbo/Music-Source-Separation-Training to write phase-aligned 48 kHz WAV stems (vocals.wav, bass.wav, drums.wav, synth.wav, sfx.wav) into the sandboxed MCP read root[2]. Next, melodic stems are routed through Dream-High/RMVPE (using 128-frame window slicing and to_local_average_cents or Viterbi decoding as implemented in RMVPE/evaluate.py) and torchcrepe to extract frame-level F_0 contours, which music21 quantizes into both Standard MIDI Files (.mid) and engraved .musicxml scores[29]. Concurrently, isolated synthesizer and tonal stems are evaluated by the Diffsynth / SSSSM-DDSP parameter estimator network—fine-tuned on target audio via auraloss.freq.MultiResolutionSTFTLoss(fft_sizes=[1024, 2048, 8192], hop_sizes=[256, 512, 2048], win_lengths=[1024, 2048, 8192], scale="mel", n_bins=128, sample_rate=48000, perceptual_weighting=True)—to emit a JSON dictionary of normalized [0.0, 1.0] synthesizer parameters mapped to SoundCraft's built-in soundcraft-dsp synth and sampler controls[9].

Work Package B: SoundCraft Engine & MCP Orchestration (crates/engine & crates/automation)

Coding agents working in Rust must extend crates/engine and crates/automation to ingest the ML sidecar artifacts deterministically[3]. Agents should implement a high-level orchestration command ai.deconstruct_and_match (modeled on PR #676 Edit > VO Cut Pipeline) that spawns or queries the Python sidecar off the real-time thread and applies the returned artifacts via an MCP batch transaction with keep_going error reporting enabled (PR #146)[15]. Specifically, the batch transaction must create multichannel or stereo audio tracks matching each separated stem's width (PR #128), import the separated WAV files without overwriting existing session media (PR #195), create instrument tracks populated with the transcribed MIDI clips, and load the built-in soundcraft-dsp synthesizer or sampler into the track's instrument slot[3]. The estimated parameter vector from Diffsynth is then written into the instrument's stored parameter state, leveraging PR #181 to ensure stored instrument parameters are pushed to the DSP instance prior to each render block[15]. Structural section boundaries and extracted chord progressions (event.extract_chords, utilizing saturating tick addition per PR #705) are written directly to the session ruler via markers.add (supporting single start or end bounds per PR #683)[20].

Work Package C: End-User UI Customization (crates/ui-egui & Hybrid

Web View)

Frontend coding agents must first execute the brand-stripping pass across docs/brand/ and apply custom visual tokens in crates/ui-egui, ensuring high-contrast legibility across both dark and light mixer themes (as refined in PR #177 and PR #720)[15]. Within soundcraft-ui-egui, agents must register a new movable, dockable panel titled AI Studio & Sound Matcher inside the shared docking layout (#167) alongside the existing Tracks and Groups, Clips, MIDI Editor, Memory Locations, and Undo History panels[3]. This panel will provide end-user controls to trigger stem separation, inspect session.audio_health diagnostics, audition candidate synthesizer patches returned by Quality-Diversity sound matching, and Save/Recall matched Track Presets (crates/engine track preset slots from PR #670 and PR #671)[3]. Furthermore, agents should expose accessibility nodes on Track and Groups list rows to resolve issue #651[19]. For deployments utilizing the WebAssembly target (apps/soundcraft-web), frontend agents must wrap the WASM canvas alongside a TypeScript OpenSheetMusicDisplay (OSMD v2.2.1) component configured with IOSMDOptions (autoResize: true, pageFormat: "Endless", RenderSlursAcrossStaves: true, and incremental renderNext() loading) so that any MIDI track exported as MusicXML from soundcraft-engine renders instantaneously as an interactive, synchronized score[3].

Work Package	Target Crates / Directories	External Repos Integrated	Verification & Acceptance Test
WP-A: ML Sidecar Service	services/ml-sidecar/ (Python 3.10+)[44]	Music-Source-Separation-Training, RMVPE, torchcrepe, Diffsynth, auraloss, music21 [cite: 21, 22, 24, 39, 44]	Unit test verifying 48 kHz WAV stem output, .mid/.musicxml generation, and [0,1] JSON synth parameters[2].
WP-B: Engine & MCP Bridge	crates/engine, crates/automation, apps/soundcraft-cli [cite: 10, 60]	storytold/soundcraft (soundcraft-dsp, soundcraft-midi)[3]	Extend crates/automation/tests/agent_tasks.rs and run cargo xtask ci with SOUNDCRAFT_NO_PREFS=1[3].
WP-C: End-User UI & Score View	crates/ui-egui, apps/soundcraft, apps/soundcraft-w	opensheetmusicdisplay (v2.2.1), elemaudio/element	Verify 120 FPS docking UI, zero docs/brand/

	eb [cite: 3]	ary [cite: 46, 48]	trademarks, and headless Chromium WASM audio playback at 48 kHz[2].

Risk Mitigation, Licensing Compliance, and Strategic Conclusions

Executing this architecture requires strict adherence to three operational and legal guardrails derived from the underlying repositories. First, engineering teams must enforce a strict license contamination quarantine. While storytold/soundcraft (MIT OR Apache-2.0), Dream-High/RMVPE (Apache-2.0), csteinmetz1/auraloss (Apache-2.0), modelscope/DiffSynth-Studio (Apache-2.0), opensheetmusicdisplay (BSD-3-Clause), and elemaudio/elementary (MIT) carry permissive licenses suitable for commercial and proprietary products[3], the algorithmic live-coding environment strudel (codeberg.org/uzu/strudel) is governed by the **GNU Affero General Public License v3 (AGPL-3.0)**[50]. Consequently, Strudel code must never be statically linked or bundled into a closed-source distribution; if TidalCycles-style pattern transformations are required inside a proprietary build, they should be implemented cleanly in Rust atop soundcraft-midi and soundcraft-time[3].

Second, trademark stripping is legally mandatory before distributing any modified binary or public web build of SoundCraft[53]. The ArtCraft name, wordmark, and visual marks in docs/brand/ are explicitly excluded from the MIT OR Apache-2.0 code license under docs/brand/LICENSE-brand.txt, and permission to use them terminates automatically upon any modification or fork distributed outside upstream pull requests[53]. Automated CI scripts must purge docs/brand/ and replace all window titles, icons, and macOS .dmg background assets prior to packaging[53].

Finally, to insulate end users from SoundCraft's pre-alpha instabilities in third-party plugin hosting—specifically the open issues surrounding VST3 0–1 parameter range display (#599), VST3 automation (#600), and macOS Audio Unit Cocoa views (#640)[19]—initial production releases of the neural synthesizer sound-matching pipeline should target SoundCraft's native, safe-Rust soundcraft-dsp synthesizers, sampler, and internal effect suite, or host external plugins exclusively through the soundcraft-clap-host CLAP interface[2]. By anchoring the user experience and real-time audio engine in storytold/soundcraft while delegating heavy neural stem separation, RMVPE pitch transcription, and differentiable spectral sound matching to decoupled Python sidecars over the Model Context Protocol, the resulting platform achieves both 120 FPS native studio responsiveness and state-of-the-art AI audio intelligence[3].

Works cited

1. https://drive.google.com/open?id=1HiqpfEdojYDrapLsljYcjdE1qya29hTTwwLs5-4hKso

2. ArtCraft SoundCraft: Free Open-Source DAW, Tested (v0.7.0), https://artcrafttutorial.com/soundcraft/
3. GitHub - storytold/soundcraft: An open-source, clean-room, https://github.com/storytold/soundcraft
4. Professional audio recording and mixing, rebuilt in pure Rust. - ArtCraft, https://getartcraft.com/apps/soundcraft
5. soundcraft - Solaire97's Gitea, https://gitea.solaire97.ovh/Solaire97/soundcraft/src/commit/4627084956afbd3fe32218e79d6d28e41f8a8f53/Cargo.toml
6. Soundcraft: An open-source, clean-room reimplementation of Avid, https://alternativeto.net/software/soundcraft/about/
7. SoundCraft: Free, Open Source Pro Tools Alternative | Opensrcs, https://opensrcs.com/project/soundcraft
8. Crafting Apps: open-source creative tools - ArtCraft, https://getartcraft.com/apps
9. Solaire97/soundcraft - soundcraft - Solaire97's Gitea, https://gitea.solaire97.ovh/Solaire97/soundcraft/src/commit/9da524fd66727fc6e42d708d5ded682ae519c9f4
10. Foundation: time, audio-io, midi, dsp, model, mix, engine, playback, https://gitea.solaire97.ovh/Solaire97/soundcraft/commit/755f02fc85d7bc33cd12bec23a10535a8f0474b3
11. CLAUDE.md - storytold/soundcraft - GitHub, https://github.com/storytold/soundcraft/blob/main/CLAUDE.md
12. soundcraft/AGENTS.md at ... - Solaire97's Gitea, https://gitea.solaire97.ovh/Solaire97/soundcraft/src/commit/7c64b277536f1bf9cb08770d17959c5a7f0b5dd6/AGENTS.md
13. soundcraft/ROADMAP.md at ... - Solaire97's Gitea, https://gitea.solaire97.ovh/Solaire97/soundcraft/src/commit/4627084956afbd3fe32218e79d6d28e41f8a8f53/ROADMAP.md
14. CLAP plugin hosting (isolated unsafe crate), stem bouncing, https://gitea.solaire97.ovh/Solaire97/soundcraft/commit/2fc8e2ecb893e1368c64f942195068519b7b4ddf
15. Releases · storytold/soundcraft - GitHub, https://github.com/storytold/soundcraft/releases
16. Soundcraft a free DAW fom Artcraft (controversial) - KVR Audio, https://www.kvraudio.com/forum/viewtopic.php?t=634178
17. AUR (en) - soundcraft - Arch Linux User Repository, https://aur.archlinux.org/packages/soundcraft
18. GitHub - storytold/soundcraft: An open-source, clean-room ... - Reddit, https://www.reddit.com/r/linuxaudio/comments/1x3frkh/github_storytoldsoundcraft_an_opensource/
19. Issues · storytold/soundcraft - GitHub, https://github.com/storytold/soundcraft/issues
20. Pull requests · storytold/soundcraft - GitHub, https://github.com/storytold/soundcraft/pulls
21. Music Source Separation Training - Context7,

https://context7.com/zfturbo/music-source-separation-training
22. SYNTHESIZER SOUND MATCHING WITH DIFFERENTIABLE DSP, https://archives.ismir.net/ismir2021/paper/000053.pdf
23. Open Sheet Music Display - GitHub, https://github.com/opensheetmusicdisplay
24. rmvpe - Dream-High - GitHub, https://github.com/Dream-High/RMVPE
25. Repository for training models for music source separation. - GitHub, https://github.com/ZFTurbo/Music-Source-Separation-Training
26. SFX isolation · Anjok07 ultimatevocalremovergui · Discussion #1287, https://github.com/Anjok07/ultimatevocalremovergui/discussions/1287
27. Audio Development Tools (ADT) - GitHub, https://github.com/Yuan-ManX/audio-development-tools
28. RMVPE: A Robust Model for Vocal Pitch Estimation in Polyphonic, https://huggingface.co/papers/2306.15412
29. RMVPE/evaluate.py at main - GitHub, https://github.com/Dream-High/RMVPE/blob/main/evaluate.py
30. maxrmorrison/torchcrepe: Pytorch implementation of the CREPE, https://github.com/maxrmorrison/torchcrepe
31. TorchCrepe pitch tracker: 2000,0000+ downloads., https://interactiveaudiolab.github.io/news/2023/10/18/TorchCrepe.html
32. A vanilla implementation of DDSP in pytorch, using torchcrepe, https://github.com/ctralie/ddsp
33. GitHub - gudgud96/torchcrepeV2: My own version of crepe, SOTA, https://github.com/gudgud96/torchcrepeV2
34. Sound Matching Using Synthesizer Ensembles, https://www.dafx.de/paper-archive/2024/papers/DAFx24_paper_76.pdf
35. Works on Synthesizer Sound Matching - Naotake Masuda, https://hyakuchiki.github.io/Diffsynth/
36. Neural Proxies for Sound Synthesizers: Learning Perceptually, https://arxiv.org/html/2509.07635v1
37. Audio Signal Processing in the Artificial Intelligence Era: Challenges, https://www.merl.com/publications/docs/TR2025-116.pdf
38. Sound Synthesis Parameters - ORBi UMONS, https://orbi.umons.ac.be/bitstream/20.500.12907/52109/1/PhD%20Thesis%20-%20Gwendal%20Le%20Vaillant%20-%20v2.pdf
39. csteinmetz1/auraloss: Collection of audio-focused loss functions in, https://github.com/csteinmetz1/auraloss
40. auraloss: Audio-focused loss functions in PyTorch, https://christian-steinmetz.squarespace.com/s/DMRN15__auraloss__Audio_focused_loss_functions_in_PyTorch.pdf
41. modelscope/DiffSynth-Studio: Enjoy the magic of Diffusion models!, https://github.com/modelscope/diffsynth-studio
42. Daily Papers - Hugging Face, https://huggingface.co/papers?q=vocal%20resynthesis
43. Paper Digest: Recent Papers on AI for Music, https://www.paperdigest.org/2020/07/recent-papers-on-ai-for-music/

44. convert musicxml files to midi using music21 - GitHub, https://github.com/ianberman/MusicXML-to-MIDI-Converter/
45. OpenSheetMusicDisplay, https://opensheetmusicdisplay.org/
46. Releases · opensheetmusicdisplay/opensheetmusicdisplay - GitHub, https://github.com/opensheetmusicdisplay/opensheetmusicdisplay/releases
47. music21.musicxml.xmlToM21, https://music21.org/music21docs/moduleReference/moduleMusicxmlXmlToM21.html
48. Offline rendering examples at the command line with Node.js · GitHub, https://github.com/elemaudio/offline-examples
49. GitHub - cristianvogel/srvb-plugin-svelte: WIP Fork of SRVB, a, https://github.com/cristianvogel/srvb-plugin-svelte
50. GitHub - tidalcycles/strudel: MOVED TO CODEBERG, https://github.com/tidalcycles/strudel
51. How TidalCycles' Pattern Algebra Was Reimagined for the Web, https://starlog.is/articles/ai-dev-tools/tidalcycles-strudel
52. tidalcycles/strudel: v1.1.0 Bananensplit | Zenodo, https://zenodo.org/records/11436758
53. soundcraft/docs/brand/LICENSE-brand.txt at main - GitHub, https://github.com/storytold/soundcraft/blob/main/docs/brand/LICENSE-brand.txt
54. GitHub - craii/strudel-coding-music: Web-based environment for live, https://github.com/craii/strudel-coding-music
55. Solaire97/soundcraft - soundcraft - Solaire97's Gitea, https://gitea.solaire97.ovh/Solaire97/soundcraft/commits/commit/ccd1f995bf128c3089e1f44b82d14b75168dbc3b/apps
56. MCP server - Solaire97's Gitea, https://gitea.solaire97.ovh/Solaire97/soundcraft/src/commit/45e32b978126f33add78b4702eaac6c5b12fe244/docs/mcp.md
57. soundcraft - Solaire97's Gitea, https://gitea.solaire97.ovh/Solaire97/soundcraft/src/commit/dba0bea03d4adc6f9d052a6d1772b15db904537e/Cargo.toml
58. Solaire97/soundcraft, https://gitea.solaire97.ovh/Solaire97/soundcraft/commits/commit/a867f05177f3a5584e4086ac61361e35a3d92f81
59. OSMDOptions.ts - opensheetmusicdisplay - GitHub, https://github.com/opensheetmusicdisplay/opensheetmusicdisplay/blob/develop/src/OpenSheetMusicDisplay/OSMDOptions.ts
60. soundcraft/apps/soundcraft-cli at ... - Solaire97's Gitea, https://gitea.solaire97.ovh/Solaire97/soundcraft/src/commit/ef47ab59d017df9d9820886805e89afdd2261788/apps/soundcraft-cli